Tokenization

What is Tokenization?

- Segmenting an input stream into an ordered sequence of tokens is called **tokenization**.

- A system which splits texts into word tokens is called a **tokenizer**

Tokenization- General Approaches?

- Based on white space

A very simple example:

✓**Input text:**

John likes Mary and Mary likes John.

✓**Tokens:**

{"John", "likes", "Mary", "and", "Mary", "likes", "John", "."}

Tokenization- General Approaches?

- Based on white space
- Based on regular expressions or rules

Regular Expressions

- Strings with a special syntax
- Allow us to match patterns in other strings
- Applications of regular expressions
  - ✓ Find all web links in a document
  - ✓ Parse email addresses
  - ✓ Remove/replace unwanted characters

Regular Expressions

pattern	matches	examples
\w+	word	'Magic'
\d	digit	9
\s	space	''
.*	wildcard	'username74'
+ or *	greedy match	'aaaaaaa'
\S	not space	'no_spaces'
[a-z]	lowercase group	'abcdfg'

Regular Expressions

pattern	matches	example
[A-Za-z]+	upper and lowercase English alphabet	'ABCDEFghijk'
[0-9]	numbers from 0 to 9	9
[A-Za-z-.]+	upper and lowercase English alphabet, - and .	'My-Website.com'
(a-z)	a, - and z	'a-z'

Regular Expressions - Example

- find cases of the English article the

  ✓ the

Regular Expressions - Example

- find cases of the English article the

  ✓ the

- this pattern will miss the word when it begins a sentence and hence is capitalized (i.e., The).

  ✓ [tT]he

Regular Expressions - Example

- find cases of the English article the

  ✓ the

- this pattern will miss the word when it begins a sentence and hence is capitalized (i.e., The).

  ✓ [tT]he

- Incorrectly return texts with the embedded in other words (e.g., other or theology).

  ✓ \b[tT]he\b

Regular Expressions - Example

- find cases of the English article the

  ✓ the

- this pattern will miss the word when it begins a sentence and hence is capitalized (i.e., The).

  ✓ [tT]he

- Incorrectly return texts with the embedded in other words (e.g., other or theology).

  ✓ \b[tT]he\b

- We might want to find the in some context where it might also have underlines or numbers nearby (the_ or the25), there are no alphabetic letters on either side of the:

  ✓ [^a-zA-Z][tT]he[^a-zA-Z]

Regular Expressions - Example

- find cases of the English article the

  ✓ the

- this pattern will miss the word when it begins a sentence and hence is capitalized (i.e., The).

  ✓ [tT]he

- Incorrectly return texts with the embedded in other words (e.g., other or theology).

  ✓ \b[tT]he\b

- We might want to find the in some context where it might also have underlines or numbers nearby (the_ or the25), there are no alphabetic letters on either side of the:

  ✓ [^a-zA-Z][tT]he[^a-zA-Z]

- It won't find the word the when it begins a line. Before the we require either the beginning-of-line or a non-alphabetic character, and the same at the end of the line:

  ✓ (^|[^a-zA-Z])[tT]he([^a-zA-Z]|$)

Regular Expressions- Example

- end with "!" or
- end with "?" or
- end with "..." or
- end with "." and the full stop is not followed by a number or
- end with "." and the full stop is followed by a whitespace
- !|\?|\.{3}|\.\D|\.\s

Regular Expressions

- Python's re Module (https://docs.python.org/3/library/re.html)
  - `split`: split a string on regex
  - `findall`: find all patterns in a string
  - `search`: search for a pattern
  - .....

Regular Expressions - Examples

- string **=** "Let's write RegEx! Won't that be fun? I sure think so. Can you find 5 sentences? Or perhaps, all 21 words?"
- sentence_endings **=** r”[\.|\?|!]”
- re**.**split**(**sentence_endings**,** string**)**

Regular Expressions - Examples

- string **=** "Let's write RegEx! Won't that be fun? I sure think so. Can you find 5 sentences? Or perhaps, all 21 words?"
- sentence_endings **=** r”[\.|\?|!]”
- re**.**split**(**sentence_endings**,** string**)**
- Output: ["Let's write RegEx", " Won't that be fun", ' I sure think so, ' Can you find 5 sentences', ' Or perhaps, all 21 words', “]

Regular Expressions - Examples

- string = "Let's write RegEx! Won't that be fun? I sure think so. Can you find 5 sentences? Or perhaps, all 21 words?"
- spaces = r"\s+"
- re.split (spaces, my_string)

Regular Expressions - Examples

- string **=** "Let's write RegEx! Won't that be fun? I sure think so. Can you find 5 sentences? Or perhaps, all 21 words?"
- spaces **=** r"\s+"
- re**.**split **(**spaces**,** my_string**)**
- Output: ["Let's", 'write', 'RegEx!', "Won't", 'that', 'be', 'fun?', 'I', 'sure', 'think', 'so.', 'Can', 'you', 'find', '5', 'sentences?', 'Or', 'perhaps,', 'all', ’21', ‘words?']

Regular Expressions (Homework)

```
string = "SOLDIER #1: Found them? In Mercea? The coconut's tropical!"

pattern1 = r'(\\w+|\\?|!)'
pattern2 = r’(\w+|#\d|\?|!)'
pattern3 = r'(#\\d\\w+\\?!)'

print (re.split(pattern1, string))
print (re.split(pattern2, string))
print (re.split(pattern3, string))
```

Chinese Tokenization

爱国人

- No spaces
- Two possible segmentations, both of them are syntactically and semantically correct
- Disambiguation can only be done with contextual information

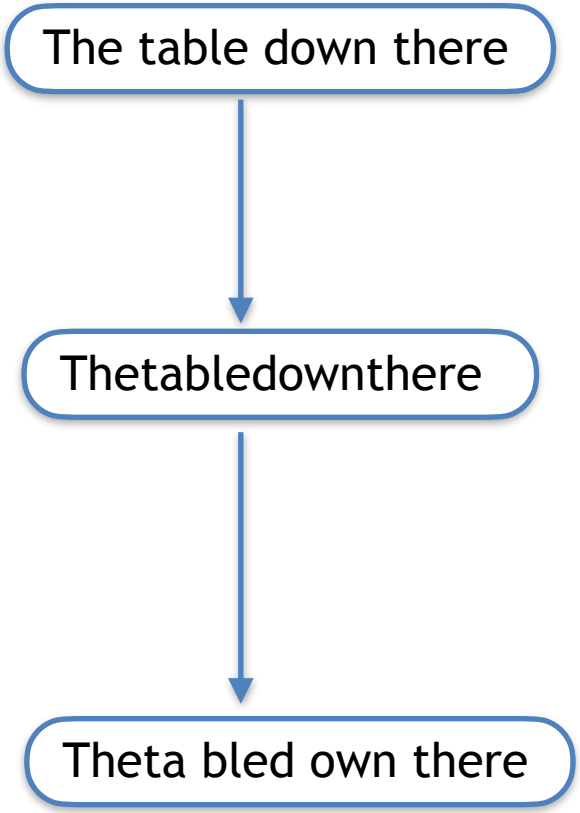
爱国 / 人

country-loving person

爱 / 国人

love country-person

Chinese Tokenization

- A greedy search called maximum matching
- The algorithm requires a dictionary (wordlist) of the language.
  - ✓ The maximum matching algorithm starts by pointing at the beginning of a string.
  - ✓ It chooses the longest word in the dictionary that matches the input at the current position.
  - ✓The pointer is then advanced past each character in that word.
  - ✓If no word matches, the pointer is instead advanced one character (creating a one-character word).
  - ✓The algorithm is then iteratively applied again starting from the new pointer position.

English Analogy